นำแนวคิด **Modern JavaScript** และ **Component-Based** มาประยุกต์ใช้ พร้อมกับการฝึกวินัยในการใช้ **Git** ตั้งแต่เริ่มต้น

Function 1.1 - The Identity Profile & Git Workflow

---

1. วัตถุประสงค์

1. เริ่มต้นโปรเจกต์ด้วยเครื่องมือสมัยใหม่ (Vite/Next.js) และตั้งค่า Version Control
2. เข้าใจการรับส่งข้อมูลผ่าน `Props` และการจัดการสถานะด้วย `useState`
3. ฝึกการใช้งาน Git ด้วยมาตรฐาน Conventional Commits

2. โจทย์โดยละเอียด

พัฒนาคอมโพเนนต์ **"Character Card"** สำหรับระบบจัดการตัวละครในเกม หรือเทียบเท่า โดยมีเงื่อนไขดังนี้:

1. **Component Structure**: สร้าง Component ที่รับข้อมูลแบบ Object ผ่าน Props ตัวอย่างเช่น ชื่อ, อาชีพ, พลังพื้นฐาน
2. **Logic** :
   - มีปุ่ม "Level Up" ที่เมื่อกดแล้วจะเพิ่มค่า "Power" แบบสุ่มในช่วง 5-15 หน่วย หรือเทียบเท่า
   - มีการคำนวณ **"Rank"** แบบไดนามิก (เช่น พลัง < 50 = Bronze, 50-100 = Silver, > 100 = Gold) หรือเทียบเท่า
   - สีของขอบการ์ดต้องเปลี่ยนตาม Rank ที่คำนวณได้ หรือเทียบเท่า
3. **Git Requirement**: ต้อง Commit อย่างน้อย 3 ครั้งตามจังหวะงานดังนี้:
   - `feat: initial project setup and component structure`
   - `feat: add level up logic and state management`
   - `style: implement dynamic ranking colors`

<u>หมายเหตุ</u> ไม่จำเป็นต้องเป็นแอปที่เกี่ยวกับตัวละครในเกม แต่สามารถสร้างแอปอะไรก็ได้ ขอให้เทียบเท่าฟังก์ชั่นเหล่านี้ แล้วตรวจตาม Checklist ด้านล่างนี้ได้ผ่านทุกข้อ

3. ขั้นตอนการปฏิบัติ (Instructions)

Step 1: Project Setup & Git First Commit

1. สร้างโปรเจกต์ด้วยคำสั่ง `npm create vite@latest lab1-app -- --template react`
2. เข้าไปในโฟลเดอร์แล้วรัน `git init`
3. สร้างคอมโพเนนต์ตาม Template และทำการ Commit ครั้งแรกด้วยข้อความ `feat: initial project setup and component structure`

Step 2: State & Random Logic

1. ใช้ `useState` โดยดึงค่าเริ่มต้นมาจาก `data.basePower`
2. เขียนฟังก์ชัน `handleLevelUp` โดยใช้ `Math.random()` เพื่อคำนวณการเพิ่มพลัง หรือเทียบเท่า
3. ทำการ Commit ครั้งที่สอง: `feat: add level up logic and state management`

Step 3: Styling & Conditional Rendering

1. เขียนฟังก์ชัน `getRank` เพื่อประเมินระดับ หรือเทียบเท่า
2. ใช้ตรรกะ Rank ในการเปลี่ยนสี `borderColor` (เช่น Gold = ทอง, Silver = เทา, Bronze = น้ำตาล) หรือเทียบเท่า
3. ทำการ Commit ครั้งสุดท้าย: `style: implement dynamic ranking colors`

💡 **คำถาม :**

- ถ้าเราไม่ใช้ `useState` แต่ใช้ตัวแปรธรรมดาเก็บค่าพลัง หน้าจอจะเปลี่ยนเลขตามที่เรากดปุ่มไหม? เพราะอะไร?
- ทำไมเราถึงส่ง `INITIAL_DATA` ผ่าน Props แทนที่จะสร้างไว้ข้างใน CharacterCard เลย?

Checklist Function 1.1

C1. Git History

- **วิธีดู:** ให้เปิด Terminal แล้วพิมพ์คำสั่ง `git log --oneline`
- **ผ่าน:** ต้องเห็นประวัติการ Commit อย่างน้อย 3 รายการ และข้อความต้องเป็นระเบียบตามที่กำหนด เช่น:
  - `feat: ...`
  - `feat: ...`
  - `style: ...`
- **จุดชี้ขาด:** หากทำเสร็จหมดแล้วค่อย `git add .` และ `git commit` รวบยอดเพียงครั้งเดียว **(ถือว่าไม่ผ่านเกณฑ์ Git)** เพราะวัตถุประสงค์คือการฝึกบันทึกงานเป็นระยะ

C2. Component Logic & State

- **วิธีดู (The Level Up Test):** ให้ลองกดปุ่ม **Level Up** รัวๆ
- **ผ่าน:**
  1. ตัวเลขพลัง (Power) ต้อง **เพิ่มขึ้นแบบไม่เท่ากัน** ในแต่ละครั้ง (เช่น +7, +12, +5) เพื่อพิสูจน์ว่าใช้ `Math.random()` จริง
  2. ตัวเลขต้องอัปเดตบนหน้าจอทันที
- **จุดชี้ขาด:** หากกดแล้วเลขไม่เปลี่ยน หรือเพิ่มทีละ 1 ตลอดเวลา แสดงว่าตรรกะ Random ผิด
- **เกณฑ์เทคนิค:** ตรวจดูว่าใช้ `useState` หรือไม่

C3. Dynamic Ranking & Style

- **วิธีดู (The Rank Transition):** ให้กดปุ่ม Level Up ไปเรื่อยๆ จนพลังแตะระดับ 50 และ 100
- **ผ่าน:**
  1. ข้อความ Rank ต้องเปลี่ยนจาก **Bronze** ➡️ **Silver** ➡️ **Gold** ตามลำดับพลังโดยอัตโนมัติ
  2. **สีขอบ (Border Color)** ของการ์ดต้องเปลี่ยนตาม Rank ทันที (เช่น พอเป็น Gold ขอบต้องกลายเป็นสีทอง)
- **จุดชี้ขาด:** ถ้า Rank เปลี่ยนแต่สีไม่เปลี่ยน หรือต้องกดปุ่มอื่นก่อนสีถึงจะเปลี่ยน แสดงว่าไม่ได้ผูกตรรกะ Rank เข้ากับ `style` ของ Component โดยตรง

<u>หมายเหตุ</u> ไม่จำเป็นต้องเป็นแอปที่เกี่ยวกับตัวละครในเกม แต่สามารถสร้างแอปอะไรก็ได้ ขอให้เทียบเท่าฟังก์ชั่นเหล่านี้ แล้วตรวจตาม Checklist เหล่านี้ได้ผ่านทุกข้อ

การจัดการ **Asynchronous JS** และการใช้ **Hooks** ในสถานการณ์จริง โดยเน้นไปที่การดึงข้อมูลจาก API และการจัดการ State ที่มีความซับซ้อนมากขึ้น

Function 1.2 - Async User Directory & Filter Logic

---

1. วัตถุประสงค์

1. เข้าใจการทำงานของ `Asynchronous JavaScript` (Async/Await) ใน React
2. ใช้งาน `useEffect` เพื่อดึงข้อมูลจาก REST API ภายนอก
3. ฝึกทักษะการทำ **Data Filtering** และจัดการสถานะการโหลด (Loading States)

2. โจทย์

หน้าเว็บแสดงรายชื่อ (User Directory) โดยมีเงื่อนไขดังนี้:

1. **API Fetching**: ต้องดึงข้อมูลจาก `https://jsonplaceholder.typicode.com/users` โดยใช้ `Async/Await`
2. **Search Logic (ระดับกลาง)**:
   - มีช่องค้นหา (Input) ที่สามารถกรองรายชื่อตาม **ชื่อ (Name)** หรือ **อีเมล (Email)** ได้แบบ Real-time
   - หากผลการค้นหาว่างเปล่า (Empty) ให้แสดงข้อความแจ้งเตือนว่า **"No results found for '[คำค้นหา]'"**
3. **Loading & Error State**:
   - ต้องแสดงข้อความหรือ Spinner ว่า **"Loading..."** ขณะกำลังดึงข้อมูล
   - ต้องจัดการ Error (ใช้ `try-catch`) หาก API ล่มหรือ URL ผิดพลาด
4. **Git Requirement**: ทำการ Commit อย่างน้อย 2 ครั้ง:
   - `feat: fetch user data from API with loading state`
   - `feat: implement search filter for name and email`

<u>หมายเหตุ</u> ไม่จำเป็นต้องเป็นแอปที่เกี่ยวกับ รายชื่อที่ตรงตามตัวอย่างนี้ แต่สามารถสร้างแอปอะไรก็ได้ ขอให้เทียบเท่าฟังก์ชั่นเหล่านี้ แล้วตรวจตาม Checklist ด้านล่างนี้ได้ผ่านทุกข้อ

3. ขั้นตอนการปฏิบัติ (Instructions)

Step 1: Fetching Data

1. ใช้ `axios` หรือ `fetch` ภายในฟังก์ชัน `fetchData`
2. อย่าลืมทดสอบสถานะ Loading โดยการหน่วงเวลา (Mock Delay) เล็กน้อยเพื่อให้เห็นว่า Spinner ทำงานจริงหรือไม่

Step 2: Filter Implementation

1. ในส่วน `filteredUsers` ให้ใช้ Method `.filter()` ของ Array
2. ใช้ `.toLowerCase()` ทั้งใน `searchTerm` และข้อมูลจาก API เพื่อให้ค้นหาได้แม่นยำ

Step 3: Handling Empty Results

1. ตรวจสอบความยาวของ `filteredUsers`
2. หากเป็น 0 ให้แสดงผลตามที่โจทย์กำหนด (ใช้ Conditional Rendering)

💡 **คำถาม**

- ทำไมต้องดึงข้อมูล API ภายใน `useEffect` ? ถ้าเรียกฟังก์ชัน `fetchData` ใน Component Body โดยตรงจะเกิดอะไรขึ้น?" (คำตอบ: จะเกิด Infinite Loop ของการ Re-render)
- หาก API คืนค่ากลับมาเป็น Array ว่าง `[]` ระบบค้นหาจะแสดงผลอย่างไร?

Checklist Function 1.2

C1. Async Logic & Error Handling

- **วิธีดู:** ให้นักศึกษาเปิดหน้าเว็บ แล้วกด **Refresh** * **ผ่าน:** ต้องเห็นข้อความ "Loading..." ปรากฏขึ้นมาชั่วขณะก่อนที่ข้อมูลจะแสดง
- **วิธีตรวจสอบ Error Handling:** ลอง **"แก้ URL ของ API ในโค้ดให้ผิด"** (เช่น เติมตัว x ต่อท้าย URL)
  - **ผ่าน:** หน้าจอต้องแสดงข้อความ Error ที่อ่านรู้เรื่อง (เช่น Error: 404) ไม่ใช่ปล่อยหน้าจอขาวโพลนหรือแอปพัง (Crash)
- **เกณฑ์เทคนิค:** ตรวจดูว่ามีการใช้ `try { ... } catch (err) { ... } finally { ... }` ครบถ้วนหรือไม่

C2. Filtering System

- **วิธีดู (The Multi-Search Test):** 1. ลองพิมพ์ชื่อคนที่มีอยู่ในลิสต์ (เช่น "Leanne") -> ต้องเจอ

  2. ลองพิมพ์ด้วย **ตัวพิมพ์เล็กทั้งหมด** (เช่น "leanne") -> ต้องเจอเท่ากัน (Case-insensitive)

  3. ลองพิมพ์ **อีเมล** ของคนนั้น (เช่น ".biz") -> ต้องกรองเจอคนที่มีอีเมลนั้นๆ
- **จุดชี้ขาด:** หากพิมพ์ตัวเล็กแล้วไม่เจอ หรือค้นหาได้เฉพาะ "ชื่อ" แต่ค้น "อีเมล" ไม่ได้ ถือว่ายังไม่บรรลุวัตถุประสงค์ของการทำ Multi-filter

C3. UX & Empty State

- **วิธีดู (The No-Result Test):** ลองพิมพ์คำมั่วๆ ที่ไม่มีในข้อมูล (เช่น "xyz123")
  - **ผ่าน:** ต้องมีข้อความเด้งขึ้นมาว่า **"No results found for 'xyz123'"** เพื่อบอกผู้ใช้ว่าระบบยังทำงานอยู่แต่ไม่เจอข้อมูล
- **การตรวจ Git:** ให้นักศึกษาใช้คำสั่ง `git log --oneline`
  - **ผ่าน:** ต้องมีการแยก Commit ระหว่างส่วนที่ดึง API (`feat: fetch...`) และส่วนที่ทำระบบค้นหา (`feat: implement search...`)

<u>หมายเหตุ</u> ไม่จำเป็นต้องเป็นแอปที่เกี่ยวกับรายชื่อที่ตรงตามตัวอย่างนี้ แต่สามารถสร้างแอปอะไรก็ได้ ขอให้เทียบเท่าฟังก์ชั่นเหล่านี้ แล้วตรวจตาม Checklist นี้ได้ผ่านทุกข้อ

💡 **คำถาม :**

1. **ถ้าลบ `[]` (Dependency Array) ออกจาก `useEffect` จะเกิดอะไรขึ้น?**
   - *คำตอบที่ถูกต้อง:* แอปจะดึงข้อมูล API วนลูปไม่รู้จบ (Infinite Loop) เพราะทุกครั้งที่ `setUsers` ทำงาน Component จะ Re-render และเรียก `useEffect` ซ้ำอีกครั้ง
2. **ทำไมถึงต้องเก็บข้อมูล Users ทั้งหมดไว้ใน State หนึ่งตัว แต่ใช้ Filter สร้างตัวแปรใหม่ (filteredUsers) แทนที่จะแก้ State users ไปเลย?**
   - *คำตอบที่ถูกต้อง:* เพราะถ้าเราไปแก้ State users ข้อมูลต้นฉบับจะหายไป เมื่อเราลบคำค้นหาออก เราจะไม่สามารถดึงรายชื่อทั้งหมดกลับมาแสดงได้เหมือนเดิม
3. **ในคำสั่ง `.filter()` ถ้าเราลบ `.toLowerCase()` ออกทั้งสองฝั่ง ผลลัพธ์จะเป็นยังไง?**
   - *คำตอบที่ถูกต้อง:* การค้นหาจะเข้มงวดขึ้น ถ้าพิมพ์ชื่อตัวเล็กแต่ในฐานข้อมูลเป็นตัวใหญ่ จะค้นหาไม่เจอทันที

การทำงานร่วมกัน (Collaboration) โดยใช้ **Git Branching Strategy** และการจัดการสถานะที่ซับซ้อนใน React

---

Function 1.3 - The Git Branching Multi-Theme Dashboard

---

1. วัตถุประสงค์

1. ฝึกการใช้ Git Branching (สร้าง, สลับ, และรวม Branch)
2. ฝึกการแก้ไข **Merge Conflict** อย่างเป็นระบบ
3. จัดการ State ระดับ Global เบื้องต้นผ่านการส่ง Props (Prop Drilling) และการจำลอง Async Configuration

2. โจทย์

ทำหน้า Dashboard ที่เปลี่ยนสีได้ (Theme) โดยต้องทำงานแยกส่วนกันเพื่อจำลองการทำงานเป็นทีม:

1. **Task A (Branch: `feature/dark-mode`):** สร้างระบบ Toggle เพื่อสลับสีพื้นหลังระหว่าง Black และ White
2. **Task B (Branch: `feature/color-picker`):** สร้างระบบเลือกสี "Primary Color" (เช่น สีของปุ่ม) โดยผู้ใช้เลือกเองได้
3. **The Conflict:** ทั้งสองงานต้องเข้ามาแก้ไขไฟล์ `ThemeContext.jsx` (หรือ `App.jsx`) พร้อมกัน ทำให้ต้องแก้ Conflict เมื่อนำมารวมกันที่ `main`

<u>หมายเหตุ</u> Final Project จะต้องเป็นแอปที่มีฟีเจอร์<u>เทียบเท่า</u>ฟังก์ชั่นเหล่านี้ แล้วตรวจตาม Checklist ด้านล่างนี้ได้ผ่านทุกข้อ

3. ขั้นตอนการปฏิบัติ (Git Workflow Mastery)

ขั้นที่ 1: แยกสายการพัฒนา (Branching)

1. สร้าง Branch แรก: `git checkout -b feature/dark-mode`
   - เขียน Logic ปุ่ม Toggle เพื่อสลับ `config.theme` ระหว่าง 'light' และ 'dark'
   - Commit งาน: `feat: add dark mode toggle`
2. สลับกลับไปที่ `main` : `git checkout main`
3. สร้าง Branch ที่สอง: `git checkout -b feature/color-picker`
   - สร้าง Input ประเภท `color` เพื่อให้ผู้ใช้เปลี่ยน `config.primaryColor`
   - Commit งาน: `feat: add dynamic primary color picker`

ขั้นที่ 2: การรวมร่างและเผชิญหน้ากับ Conflict

1. สลับไปที่ `main` แล้วรวม Branch แรก:
   - `git merge feature/dark-mode` (ตรงนี้จะผ่านฉลุย)
2. รวม Branch ที่สอง:
   - `git merge feature/color-picker`
   - **! STOP:** ตรงนี้ Git จะแจ้งว่า **Merge Conflict** ให้ใช้ VS Code เปิดไฟล์ขึ้นมาแล้วตัดสินใจว่าจะ "เอาโค้ดของทั้งสอง Branch มาวางต่อกันอย่างไรให้โปรแกรมทำงานได้ครบทั้ง 2 ฟีเจอร์"

ขั้นที่ 3: สรุปงาน

1. เมื่อแก้ Conflict เสร็จแล้ว ให้ `git add .` และ `git commit -m "chore: resolve conflict between dark-mode and color-picker"`
2. ตรวจสอบว่า Dashboard สามารถ "เปลี่ยนโหมดมืด" พร้อมกับ "เปลี่ยนสีปุ่ม" ได้ในเวลาเดียวกันหรือไม่

💡 **คำถาม**

- ถ้า Merge `feature/color-picker` เข้าไปใน `feature/dark-mode` ก่อน แล้วค่อย Merge เข้า `main` จะเกิดอะไรขึ้น? (คำตอบ: Conflict จะไปเกิดที่ Branch ลูกแทน ซึ่งเป็นวิธีที่นิยมในทีมใหญ่เพื่อรักษา `main` ให้สะอาด)
- "ในตอนที่แก้ Conflict ทำไมถึงไม่ควรลบโค้ดของเพื่อนทิ้ง?" (คำตอบ: เพื่อรักษา Integrity ของฟีเจอร์ที่พัฒนาขนานกันมา)

Checklist Function 1.3

C1. Git Branching Strategy

- **วิธีดู (The Tree Test):** ให้นักศึกษาพิมพ์คำสั่ง `git log --graph --oneline --all` ใน Terminal
- **ผ่าน:** ต้องเห็นเส้นกราฟที่ **"แยกออกเป็นสองง่าม"** จากจุดเดียวกัน และกลับมาบรรจบกันที่ `main` ในภายหลัง
- **จุดชี้ขาด:** หากเห็นเป็นเส้นตรงเส้นเดียว แสดงว่าไม่ได้สร้าง Branch จริง (ใช้การลบโค้ดเก่าเขียนทับ)
    - หากไม่มีการแตกแขนง แสดงว่าไม่ได้ทำตามขั้นตอน Git Workflow

C2. Conflict Resolution Mastery

- **วิธีดู (The Hybrid UI):** ดูที่หน้าจอ Dashboard หลังจาก Merge เสร็จแล้ว
- **ผ่าน:** ต้องเห็น **ทั้งปุ่มสลับ Dark Mode** และ **ช่องเลือกสี (Color Picker)** อยู่บนหน้าจอเดียวกัน
- **จุดชี้ขาด:** หากฟีเจอร์ใดฟีเจอร์หนึ่งหายไป (เช่น มีปุ่มมืดแต่ช่องเลือกสีหาย) แสดงว่าตอนแก้ Conflict ใช้วิธี "Accept Theirs" หรือ "Accept Ours" คือเลือกเอาโค้ดของฝั่งใดฝั่งหนึ่งแล้วลบของอีกฝั่งทิ้ง
    - **สิ่งที่ถูกต้อง:** ต้องเลือก "Accept Both Changes" หรือรวมโค้ดด้วยมือเพื่อให้ทำงานได้ทั้งคู่

C3. State Integrity & Preservation

- **วิธีดู (The Persistence Test):**
    1. ลองเปลี่ยนสีปุ่มเป็น "สีแดง"
    2. จากนั้นลองกดสลับเป็น "Dark Mode"
- **ผ่าน: ปุ่มต้องยังเป็นสีแดงอยู่** ไม่กลับไปเป็นสีฟ้าตั้งต้น
- **เกณฑ์เทคนิค:** ตรวจดูว่าใช้ `setConfig(prev => ({ ...prev, theme: 'dark' }))` หรือไม่
    - หากเขียนแค่ `setConfig({ theme: 'dark' })` ค่า `primaryColor` จะหายไปทันที (State Overwrite) นี่คือจุดที่วัดว่าเข้าใจความหมายของสเตทใน React หรือไม่

<u>หมายเหตุ</u> Final Project จะต้องเป็นแอปที่มีฟีเจอร์<u>เทียบเท่า</u>ฟังก์ชั่นเหล่านี้ แล้วตรวจตาม Checklist ได้ผ่านทุกข้อ

💡 **คำถาม :**

1. **ตอนเกิด Conflict ใน VS Code สีเขียว (Current Change) กับสีน้ำเงิน (Incoming Change) ต่างกันยังไง?**
    - *คำตอบที่ถูกต้อง:* "สีเขียวคือโค้ดที่อยู่บน Branch ที่อยู่นิ่งๆ (main) ส่วนสีน้ำเงินคือโค้ดที่กำลังพยายามดึงเข้ามาควบรวม
2. **ทำไมเราถึงไม่ควรแก้ Conflict โดยการลบโค้ดที่มีเครื่องหมาย `<<<<<<<` และ `>>>>>>>` ออกเฉยๆ โดยไม่อ่านโค้ดข้างใน?**
    - *คำตอบที่ถูกต้อง:* "เพราะเครื่องหมายเหล่านั้นแค่บอกจุดที่โค้ดชนกัน ถ้าลบออกโดยไม่จัดระเบียบโค้ดใหม่ โปรแกรมจะเกิด Syntax Error และทำงานไม่ได้

3. **ถ้าเราทำงานทีมใหญ่ แล้วเพื่อนแก้ไฟล์เดียวกันกับเรา 10 คน คิดว่า Branching จะช่วยอะไรได้?**
    - *คำตอบที่ถูกต้อง:* "ช่วยให้แต่ละคนทำงานในพื้นที่ตัวเองได้โดยไม่กวนกัน และเราสามารถ Review โค้ดของแต่ละคนก่อนจะยอมให้รวมเข้าเครื่องกลางได้"

การใช้ **Recursive Component** (คอมโพเนนต์ที่เรียกตัวเอง) ซึ่งเป็นเทคนิคระดับสูงในการจัดการกับโครงสร้างข้อมูลที่ซับซ้อนและมีความลึกไม่แน่นอน เช่น ระบบคอมเมนต์ซ้อนกัน หรือโครงสร้างโฟลเดอร์

Function 1.4 - The Dynamic Recursive Comment System

---

1. วัตถุประสงค์

1. เข้าใจและประยุกต์ใช้แนวคิด **Recursion** ในการสร้าง React Component
2. ฝึกการจัดการโครงสร้างข้อมูลแบบ **Nested Objects/Arrays**

2. โจทย์

พัฒนาหน้าแสดงผล "กระทู้สนทนา" ที่สมาชิกสามารถตอบกลับ (Reply) คอมเมนต์ของกันและกันได้ไม่จำกัดชั้น โดยมีเงื่อนไขดังนี้:

1. **Recursive Rendering**: ต้องสร้าง Component ชื่อ `CommentItem` ที่สามารถเรียกตัวเองได้เพื่อแสดงผล Reply ที่ซ้อนอยู่ข้างใน
2. **State Management** : เมื่อกดปุ่ม "Add Reply" ในคอมเมนต์ใดๆ ระบบต้องสามารถแทรกข้อมูลคอมเมนต์ใหม่ลงไปในชั้นที่ถูกต้องของ Data Structure ได้

<u>หมายเหตุ</u> Final Project จะต้องเป็นแอปที่มีฟีเจอร์<u>เทียบเท่า</u>ฟังก์ชั่นเหล่านี้ แล้วตรวจตาม Checklist ด้านล่างนี้ได้ผ่านทุกข้อ

3. ขั้นตอนการปฏิบัติ (Git & Code Challenge)

Step 1: โครงสร้าง Recursive

1. พัฒนาคอมโพเนนต์ `CommentItem` ให้สามารถ Render ตัวเองซ้ำลงไปในช่อง `replies` ได้ (ระวังเรื่อง Infinite Loop)

Step 2: Logic การเพิ่มคอมเมนต์ซ้อน

1. เขียนฟังก์ชัน `addReply` โดยใช้แนวคิด Recursion ในการเดินลึกเข้าไปใน Array (Deep update)
2. ทดสอบโดยการลอง Reply ซ้อนกันอย่างน้อย 4 ชั้น

💡 **คำถาม :**

- ทำไมการแก้ข้อมูลใน Recursive Structure ถึงต้องคืนค่าเป็น Object ใหม่ (`{...c}`) แทนที่จะใส่ข้อมูลลงไปใน `c.replies.push()` ตรงๆ?" (คำตอบ: เพื่อรักษา Immutability ของ React State ให้เกิดการ Re-render)
- ในการทำ Recursive Component ต้องระวังเรื่องอะไรมากที่สุด?" (คำตอบ: การกำหนด Base Case หรือเงื่อนไขหยุด เพื่อไม่ให้เกิด Stack Overflow)

Checklist Function 1.4

C1. Recursive Component Implementation

- **วิธีดู (The Depth Test):** ให้ลองกดปุ่ม **Reply** ต่อกันไปเรื่อยๆ จนถึงชั้นที่ 5 หรือ 6
- **ผ่าน:** 1. รายการคำตอบต้องปรากฏขึ้นใต้คอมเมนต์แม่ (Parent) ของมันพอดี

  2. หน้าตา (UI) ต้องมีการ **"เยื้องขวา" (Indentation)** เข้าไปเรื่อยๆ ตามลำดับความลึก (เพื่อยืนยันว่า Component เรียกตัวเองซ้ำในที่ที่ถูกต้อง)
- **จุดชี้ขาด:** หากตอบกลับคอมเมนต์ชั้นที่ 2 แล้วข้อความไปโผล่ที่ชั้นบนสุด (Root) หรือไม่ออกมาเลย แสดงว่าเรียกใช้ Component ตัวมันเองผิดตำแหน่ง

C2. Recursive State Logic

- **วิธีดู (The Deep Insertion Test):** ให้ตอบกลับคอมเมนต์ที่อยู่ **"ลึกที่สุด"**
- **ผ่าน:** เมื่อกด Send แล้ว ข้อความใหม่ต้องแทรกเข้าไปอยู่ในกล่องนั้นทันที โดยที่ข้อความอื่นๆ ในหน้าจอห้ามหายไป
- **จุดชี้ขาด (Logic Check):** ตรวจสอบฟังก์ชัน `updateComments` ในโค้ด
  - ต้องใช้แนวคิด **Deep Update** (วนลูปเข้าไปข้างใน)
  - **ห้ามใช้** `comments.push()` หรือการแก้ไขตัวแปรเดิม (Mutation) เพราะจะทำให้ React ไม่ Re-render หรือทำงานผิดพลาด
  - ต้องมีการคืนค่าเป็น **Object ใหม่** (`return { ...c, replies: ... }`)

C3. Git Stash Workflow

- **วิธีดู (The Time-Travel Test):** เปิด Terminal แล้วพิมพ์ `git log --oneline --graph --all`
- **ผ่าน:** 1. ต้องมี Commit ที่ชื่อว่า `docs: update readme with student info` ปรากฏอยู่ในประวัติ

  2. **จุดตัดสิน:** โชว์คำสั่ง `git stash list` (ถ้ายังไม่ได้ `pop` ออกมา) หรือ **"อธิบาย"** ว่าทำไมต้องใช้ `stash` แทนที่จะ `commit` งานที่ทำค้างไว้
- **เหตุผลที่ต้องผ่าน:** ต้องเข้าใจว่าการ Commit งานที่ "ยังคอมไพล์ไม่ผ่าน" หรือ "ทำไม่เสร็จ" ลงไปในประวัติเป็นเรื่องไม่ดี การ `stash` จึงเป็นทางออกที่มืออาชีพใช้

<u>หมายเหตุ</u> Final Project จะต้องเป็นแอปที่มีฟีเจอร์<u>เทียบเท่า</u>ฟังก์ชั่นเหล่านี้ แล้วตรวจตาม Checklist นี้ได้ผ่านทุกข้อ

💡 คำถาม :

1. **ถ้าลบ `key={reply.id}` ออกจาก Recursive Component จะเกิดอะไรขึ้น?**
   - *คำตอบที่ถูกต้อง:* React จะเตือน Error และประสิทธิภาพจะตกลง เพราะ React จะแยกแยะไม่ออกว่าคอมเมนต์ไหนเปลี่ยนไป ทำให้ต้องวาดใหม่หมดทั้งต้นไม้ (Tree)
2. **ทำไมในฟังก์ชัน `updateComments` เราถึงต้องมีเงื่อนไข `if (c.replies.length > 0)` ?**
   - *คำตอบที่ถูกต้อง:* "เพื่อบอกให้โปรแกรม 'เดินลึกเข้าไปอีก' เมื่อเจอกล่องที่มีคำตอบซ่อนอยู่ข้างใน ถ้าไม่เช็ค เราจะหาคอมเมนต์ที่อยู่ชั้นลึกๆ ไม่เจอ
3. **ถ้าเผลอ `git stash` ไปแล้ว 3 รอบ แต่ลืมไปว่างานที่ต้องการอยู่ใบไหน จะหาเจอดังยังไง?**
   - *คำตอบที่ถูกต้อง:* "ใช้ `git stash list` เพื่อดูรายการทั้งหมด และใช้ `git stash apply stash@{index}` เพื่อดึงใบที่ต้องการกลับมา

การควบคุม **Asynchronous Flow** ขั้นสูง ผสมผสานกับการจัดการ **State** ที่ซับซ้อน และการใช้ **Git Collaboration Workflow** แบบเดียวกับที่ใช้ในบริษัทระดับโลก

Function 1.5 - The Async Task Orchestrator

---

1. วัตถุประสงค์

1. ฝึกการจัดการ Asynchronous Tasks แบบขนาน (Parallel) โดยมีการจำกัดจำนวน (Concurrency Limit)
2. ฝึกการทำ **Advanced State Update** เพื่อแสดง Progress Bar แยกแต่ละรายการ
3. ฝึกกระบวนการ **Fork, Branch, and Pull Request (PR)** บน GitHub

2. โจทย์

ระบบ "อัปโหลดไฟล์" หรือ "ประมวลผลรูปภาพ" 10 งาน แต่ทรัพยากรเครื่องจำกัด ทำให้รันได้ **พร้อมกันเพียงทีละ 2 งาน (Max Parallel = 2)** เท่านั้น

1. **The Queue Logic**: เมื่อกดเริ่มทำงาน งานที่ 1 และ 2 ต้องเริ่มทันที งานที่เหลือต้องสถานะเป็น "Waiting"
2. **Dynamic Progress**: เมื่อแต่ละงานเริ่มรัน ให้จำลอง Progress (0% ถึง 100%) โดยใช้ `setInterval`
3. **The Hand-off**: เมื่องานใดงานหนึ่งเสร็จสิ้น (100%) งานถัดไปในคิวต้องถูกดึงมาประมวลผลต่อทันทีจนกว่าจะครบ 10 งาน
4. **Git Challenge (GitHub Flow)**: ให้นักศึกษาทำการ **Fork** โปรเจกต์จาก Link สร้าง Branch ชื่อ `feat/orchestrator` และส่ง **Pull Request** กลับมาพร้อมคำอธิบายเชิงเทคนิค

<u>หมายเหตุ</u> Final Project จะต้องเป็นแอปที่มีฟีเจอร์<u>เทียบเท่า</u>ฟังก์ชั่นเหล่านี้ แล้วตรวจตาม Checklist ด้านล่างนี้ได้ผ่านทุกข้อ

3. ขั้นตอนการปฏิบัติ (Git Collaboration)

Step 1: GitHub Workflow

1. ให้นิสิต **Fork** Repository
2. Clone ลงเครื่อง และสร้าง Branch ใหม่: `git checkout -b feat/orchestrator`

Step 2: Coding the Orchestrator

1. เขียนฟังก์ชัน `processTask` ที่คืนค่าเป็น `Promise` เพื่อให้ระบบรู้ว่างานเสร็จเมื่อไหร่
2. ใช้ `useEffect` เป็นตัวคุม **Semaphore** (ตัวนับงานขนาน) เพื่อดึงงานออกจากคิวอัตโนมัติ

Step 3: Pull Request

1. Push โค้ดขึ้น GitHub ของตนเอง
2. กดสร้าง **Pull Request (PR)** กลับมาที่ Repository
3. ในช่องคำอธิบาย PR ให้สรุปว่า **"คุณจัดการลำดับการเรียกงาน (Execution Order) อย่างไรไม่ให้เกิน 2 งานพร้อมกัน?"**

💡 **คำถาม :**

- ทำไมการจัดการ Queue แบบนี้ถึงดีกว่าการใช้ `Promise.all(tasks)` ตรงๆ? (คำตอบ: `Promise.all` จะรันงานทั้งหมดพร้อมกัน ซึ่งถ้ามี 1,000 งาน อาจทำให้ Browser หรือ Server ล่มได้ การคุม Concurrency ช่วยเรื่อง Performance และ Reliability)
- หากงานที่ 2 เกิด Error ระหว่างทาง (Reject) ระบบจะหยุดรันงานที่เหลือ หรือจะข้ามไปทำสีอื่นต่อ? (คำตอบที่คาดหวัง: ควรข้ามไปทำตัวถัดไปเพื่อให้งานไม่ค้าง)

Checklist Function 1.5

C1. Concurrency Control

- **วิธีดู (The Parallel Test):** ให้กดปุ่ม **Start Processing**
- **ผ่าน:**
  1. ในวินาทีแรกที่กด **ต้องมีงานเพียง 2 งานเท่านั้น** ที่สถานะเปลี่ยนเป็น `running` และแถบ Progress เริ่มขยับ
  2. งานที่ 3 ถึง 10 ต้องค้างอยู่ที่สถานะ `pending` หรือ `waiting`
  3. เมื่องานใดงานหนึ่ง (เช่น Task 1) โหลดถึง 100% แล้ว **งานถัดไป (Task 3) ต้องเริ่มทำงานทันทีโดยอัตโนมัติ**
- **จุดชี้ขาด:** หากกดปุ่มแล้วงานทั้ง 10 งานเริ่มโหลดพร้อมกัน หรือต้องกดปุ่มซ้ำเพื่อให้งานถัดไปเดินต่อ แสดงว่าระบบ Queue และการคุม Concurrency ล้มเหลว

C2. State & Progress Integrity

- **วิธีดู (The Real-time UI):** สังเกตการเคลื่อนที่ของแถบ Progress Bar
- **ผ่าน:**
  1. แถบ Progress ของแต่ละ Task ต้องขยับแยกจากกัน (เช่น Task 1 อาจจะเร็วกว่า Task 2)
  2. เมื่อ Progress ครบ 100% ตัวเลขต้องหยุดนิ่ง และสถานะต้องเปลี่ยนเป็น `completed` (อาจเปลี่ยนสีแถบเป็นสีเขียวเพื่อความชัดเจน)
- **เกณฑ์เทคนิค:** ตรวจดูว่าใช้ `setTasks` โดยการ Copy State เดิม (`...prevTasks`) หรือไม่ เพื่อป้องกันข้อมูลงานอื่นหายระหว่างอัปเดตงานตัวเดียว

C3. GitHub Collaboration & PR

- **วิธีดู (The Pro Workflow):** เข้าไปดูที่หน้า **Pull Request** ใน GitHub Repository หลัก
- **ผ่าน:**
  1. มีรายการ PR จากนักศึกษาที่ส่งมาจาก **Forked Repository** ของตนเอง
  2. ในหน้า PR ต้องเขียน **"Description"** อธิบายตรรกะที่ใช้ (เช่น ใช้ `useEffect` ดักจับ `runningCount` หรือใช้ `while loop` ร่วมกับ `async`)
  3. **Commit History:** ต้องไม่เป็น `Commit 1` , `Update` , `Final` แต่ควรเป็น `feat: implement queue logic` , `fix: progress bar glitch`
- **จุดชี้ขาด:** หากใช้วิธีส่งไฟล์ `.zip` หรือส่งแค่ลิงก์หน้าเว็บโดยไม่มี PR ถือว่าไม่ผ่านเกณฑ์การทำงานร่วมกันแบบทีม

<u>หมายเหตุ</u> Final Project จะต้องเป็นแอปที่มีฟีเจอร์<u>เทียบเท่า</u>ฟังก์ชั่นเหล่านี้ แล้วตรวจตาม Checklist นี้ได้ผ่านทุกข้อ

💡 **คำถาม :**

1. **ทำไมเราต้องใช้ `useEffect` โดยใส่ `[tasks, runningCount]` เป็น Dependency?**
   - *คำตอบที่ถูกต้อง:* "เพื่อให้ React คอยตรวจสอบทุกครั้งที่งานหนึ่งเสร็จ (สถานะเปลี่ยน) หรือจำนวนงานที่รันอยู่ลดลง ระบบจะได้รู้ว่าถึงเวลาดึงงานถัดไปจากคิวมาทำแล้ว
2. **ถ้าเปลี่ยน Max Parallel จาก 2 เป็น 5 เธอต้องแก้โค้ดกี่จุด?**
   - *คำตอบที่ถูกต้อง:* "ควรแก้จุดเดียวครับ ถ้าวางโครงสร้างดีๆ โดยใช้ตัวแปร Constant กำหนดค่า Max เอาไว้ตั้งแต่แรก

3. **ในฟังก์ชัน `processTask` จัดการยังไงให้ React รู้ว่างานเสร็จแล้วก่อนจะไปเริ่มงานใหม่?**
    - *คำตอบที่ถูกต้อง:* "ใช้ `await` หรือ `Promise` เพื่อรอให้การ `setInterval` ทำงานจนครบ 100% แล้วค่อยสั่งลดค่า `runningCount` หรืออัปเดตสถานะเป็น `completed`